

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5
ПО ДИСЦИПЛИНЕ «ТИПЫ И СТРУКТУРЫ ДАННЫХ»

Тема: Обработка очередей

Вариант № 3

Студент: **Чернышева А.П.**

Группа: **ИУ7-36Б**

Преподаватель: **Силантьева А. В.**

2025 г.

Описание задачи

Требуется смоделировать процесс обслуживания до выхода из ОА2 первых 1000 заявок. Выдавать на экран после обслуживания в ОА2 каждых 100 заявок информацию о текущей и средней длине каждой очереди, а в конце процесса общее время моделирования, время простоя ОА2, количество срабатываний ОА1, среднее времени пребывания заявок в очереди. Обеспечить по требованию пользователя выдачу на экран адресов элементов очереди при удалении и добавлении элементов. Проследить, возникает ли при этом фрагментация памяти. Заявки поступают в "хвост" каждой очереди; в ОА они поступают из "головы" очереди по одной и обслуживаются по случайному закону за интервалы времени Т1 и Т2, равномерно распределенные от 0 до 6 и от 1 до 8 единиц времени для ОА1 и ОА2 соответственно. (Все времена – вещественного типа). Каждая заявка после ОА1 с вероятностью P=0.7 вновь поступает в "хвост" первой очереди, совершая новый цикл обслуживания, а с вероятностью 1-P входит во вторую очередь. В начале процесса все заявки находятся в первой очереди.

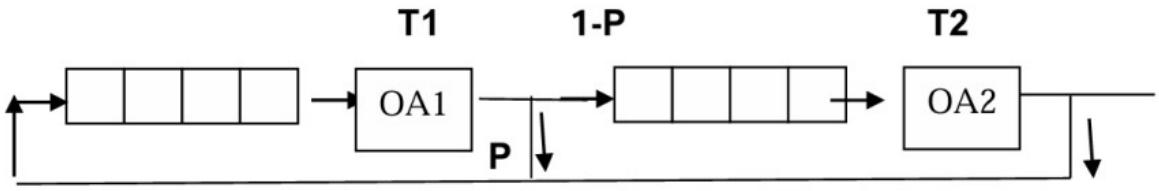


Рис. 1 Схема очереди

Описание ТЗ

Описание исходных данных и результатов

Входные данные:

Введенный пункт меню -- целое число от 0 до 4

Меню:

1) Провести симуляцию процесса обслуживания очереди на массиве

2) Провести симуляцию процесса обслуживания очереди на списке

3) Изменить параметры симуляции

4) Вывести результат замеров удаления и добавления

0) Завершить работу программы

Параметры обслуживания (при изменении):

- Начальное время обработки ОА1: вещественное число не меньше 0
- Конечное время обработки ОА1: вещественное число не меньше начального значения
- Начальное время обработки ОА2: вещественное число не меньше 0
- Конечное время обработки ОА2: вещественное число не меньше начального значения
- Вероятность возврата заявки в первую очередь: вещественное число от 0 до 1

Ввод элемента в очередь(целое число)

Максимальный размер очереди: MAX_QUEUE_SIZE = 100 элементов

Результаты:

1. Результат процесса обслуживания очереди
   - Общее время работы системы (вещественное число)
   - Теоретическое расчетное время (вещественное число)
   - Процент погрешности между фактическим и теоретическим временем (вещественное число)
   - Время простоя второго аппарата (ОА2) (вещественное число)
   - Количество срабатываний первого аппарата (ОА1) (целое число)
   - Средняя длина первой очереди (вещественное число)

- Средняя длина второй очереди (вещественное число)

2. Таблица замеров времени и памяти
3. Вывод освобожденных областей памяти

Описание задачи, реализуемой программой

Программа предназначена для реализации алгоритмов работы с очередью и сравнительной оценки их эффективности (времени выполнения операций и использования памяти) между массивом и списком. Сравнение проводится при различном количестве элементов в очереди и различных операциях работы с ней.

Способ обращения к программе

Программа запускается через исполняемый файл app.exe.

Описание возможных аварийных ситуаций и ошибок пользователя

1. Ошибка при вводе невалидных данных (буквы вместо чисел)
2. Ошибка переполнения очереди
3. Ошибка выделения памяти при работе со списком
4. Ввод вероятности вне диапазона [0, 1]
5. Ввод отрицательного времени обработки
6. Ввод некорректного диапазона времени (конечное время меньше начального)
7. Попытка выполнения операций с неинициализированной очередью

Описание внутренних структур данных

Хранение информации об отдельной заявке в системе обслуживания.

```
typedef struct
{
    int id;
    double entered_time;
} request_t;
```

1. id — номер заявки

2. entered_time — время вхождения в ОА

Структура очереди на массиве

```
typedef struct
{
    request_t data[MAX_ARR_SIZE];
    int size;
    int max_size;
    int head;
    int tail;
} arr_queue_t;
```

1. data[MAX_ARR_SIZE] — элементы очереди, MAX_ARR_SIZE = 500 элементов
2. size — размер очереди
3. max_size — максимальный размер очереди
4. head — индекс вершины
5. tail -индекс хвоста

Структура узла списка

```
typedef struct node
{
    request_t data;
    struct node *next;
} node_t;
```

1. data — элементы очереди
2. *next — указатель на следующий элемент

Структура узла списка свободной памяти

```
typedef struct free_node
{
    void *address;
    struct free_node *next;
} free_node_t;
```

1. *addr — адрес освобожденного блока памяти

2. *next — указатель на следующую запись в списке свободных адресов

Структура очереди на списке

```
typedef struct
{
    node_t *head;
    node_t *tail;
    int size;
    int max_size;
    free_node_t *free_list;
    int free_elems_size;
} list_queue_t;
```

1. *head — указатель на голову
2. *tail — указатель на хвост
3. size — размер очереди
4. max_size — максимальный размер очереди
5. *free_list — список свободных областей памяти
6. free_elems_size — количество освобожденных элементов

Структура обслуживающего аппарата

```
typedef struct
{
    int is_busy;
    double end_time;
    request_t current_request;
} service_unit_t;
```

1. is_busy — занят ли аппарат
2. end_time — время окончания обработки заявки
3. current_request — текущая заявка в аппарате

Тип очереди

```
typedef enum {
   ARRAY_QUEUE,
   LIST_QUEUE
} queue_type_t;
```

1. ARRAY_QUEUE — очередь реализована на массиве.
2. LIST_QUEUE — очередь реализована на списке.

Общий алгоритм программы

Программа реализует систему с двумя очередями и двумя обслуживающими аппаратами. Начальное состояние системы характеризуется наличием 100 заявок в первой очереди. Процесс моделирования продолжается до момента обработки 1000 заявок во втором аппарате.

1. Инициализация системы
    - Первая очередь (queue1) заполняется 100 начальными заявками
    - Вторая очередь (queue2) инициализируется как пустая
    - Оба обслуживающих аппарата (ОА1 и ОА2) устанавливаются в состояние "свободен"
    - Устанавливается начальное время обслуживания (current_time = 0)
2. Работа первого обслуживающего аппарата (ОА1)
    - Проверяется условие: аппарат свободен и первая очередь не пуста
    - При выполнении условий:
        - Извлекается заявка из начала первой очереди
        - Аппарат переводится в состояние "занят"
        - Заявка назначается текущей для обработки
        - Вычисляется время обслуживания как случайная величина в заданном диапазоне [t1_start, t1_end]
        - Устанавливается время завершения обработки заявки
3. Работа второго обслуживающего аппарата (ОА2)
    - Проверяется условие: аппарат свободен и вторая очередь не пуста
    - При выполнении условий:
        - Извлекается заявка из начала второй очереди
        - Аппарат переводится в состояние "занят"
        - Заявка назначается текущей для обработки
        - Вычисляется время обслуживания как случайная величина в заданном диапазоне [t2_start, t2_end]
        - Устанавливается время завершения обработки заявки

4. Определение ближайшего события завершения обработки
    - Если оба аппарата находятся в состоянии "занят", определяется аппарат с минимальным временем завершения обработки
    - Если занят только один аппарат, следующее событие связывается с его временем завершения
    - Вычисленное время устанавливается как следующее событие в системе
5. Обновление временных параметров и учет простоя
    - Если второй аппарат свободен, вычисляется интервал его простоя и добавляется к суммарному времени простоя
    - Текущее время системы обновляется до времени следующего события завершения обработки
6. Обработка завершенных заявок
    - Для первого аппарата (ОА1):
        - Генерируется случайное число для определения направления заявки
        - С заданной вероятностью p заявка возвращается в первую очередь, в противном случае направляется во вторую очередь
        - Аппарат переводится в состояние "свободен"
    - Для второго аппарата (ОА2):
        - Обработанная заявка помещается в первую очередь
        - Аппарат переводится в состояние "свободен"
        - Увеличивается счетчик обработанных заявок
7. Сбор статистических данных
    - На каждом сотом обработанной заявке фиксируются:
        - Текущая длина первой очереди
        - Текущая длина второй очереди
        - Вычисляются средние значения длин очередей

Описание алгоритмов функций

arr_queue_init(arr_queue_t *queue, int max_size) - инициализация очереди на массиве:

1. Установка размера очереди в 0
2. Установка максимального размера очереди в заданное значение
3. Установка индекса головы и хвоста в 0

is_arr_queue_empty(arr_queue_t *queue) - проверка очереди-массива на пустоту:
1. Сравнение текущего размера очереди с 0
2. Возврат 1, если очередь пуста, иначе 0

is_arr_queue_full(arr_queue_t *queue) - проверка очереди-массива на переполнение:
1. Сравнение текущего размера очереди с максимальным размером
2. Возврат 1, если очередь полна, иначе 0

arr_enqueue(arr_queue_t *queue, request_t elem) - добавление элемента в очередь-массив:
1. Проверка, что очередь не переполнена
2. Запись элемента в позицию хвоста
3. Обновление индекса хвоста с учетом кольцевого буфера (tail = (tail + 1) % max_size)
4. Увеличение счетчика размера

arr_dequeque(arr_queue_t *queue) - удаление элемента из очереди-массива:
1. Проверка, что очередь не пуста
2. Сохранение удаляемого элемента из позиции головы
3. Обновление индекса головы с учетом кольцевого буфера (head = (head + 1) % max_size)
4. Уменьшение счетчика размера
5. Возврат удаленного элемента

print_arr_queue(arr_queue_t *queue) - вывод очереди-массива:

1. Проверка, что очередь не пуста
2. Последовательный вывод элементов от головы до хвоста с учетом кольцевого буфера
3. Использование формулы (head + i) % max_size для получения индекса элемента

list_queue_init(list_queue_t *queue, int max_size) - инициализация очереди на списке:
1. Установка указателей head и tail в NULL
2. Установка размера очереди в 0
3. Установка максимального размера очереди
4. Инициализация списка свободной памяти (free_list = NULL, free_elems_size = 0)

is_list_queue_empty(list_queue_t *queue) - проверка очереди-списка на пустоту:
1. Сравнение текущего размера очереди с 0
2. Возврат 1, если очередь пуста, иначе 0

is_list_queue_full(list_queue_t *queue) - проверка очереди-списка на переполнение:
1. Сравнение текущего размера очереди с максимальным размером
2. Возврат 1, если очередь полна, иначе 0

list_enqueue(list_queue_t *queue, request_t elem) - добавление элемента в очередь-список:
1. Проверка, что очередь не переполнена
2. Выделение памяти для нового узла
3. Запись данных элемента в узел
4. Установка указателя next нового узла в NULL
5. Если очередь пуста, установка head и tail на новый узел

6. Иначе добавление узла в конец списка и обновление tail

7. Увеличение счетчика размера

8. При включенном флаге show_memory_operations вывод адреса выделенной памяти

list_dequeue(list_queue_t *queue, request_t *removed_elem) - удаление элемента из очереди-списка:

1. Проверка, что очередь не пуста

2. Сохранение данных головного узла

3. Сохранение адреса удаляемого узла в список свободной памяти

4. Освобождение памяти головного узла

5. Перемещение указателя head на следующий узел

6. Если очередь стала пустой, установка tail в NULL

7. Уменьшение счетчика размера

8. При включенном флаге show_memory_operations вывод адреса освобожденной памяти

print_list_queue(list_queue_t *queue) - вывод очереди-списка:

1. Проверка, что очередь не пуста

2. Последовательный обход списка от head до tail

3. Для каждого узла вывод адреса памяти и значения элемента

print_free_area(list_queue_t *queue) - вывод списка свободных областей:

1. Проверка, что список свободной памяти не пуст

2. Последовательный обход списка free_list

3. Для каждого элемента вывод адреса освобожденной памяти

free_list_queue(list_queue_t *queue) - освобождение памяти очереди-списка:

1. Последовательный обход очереди от head до tail

2. Для каждого узла освобождение памяти

3. Последовательный обход списка свободной памяти free_list

4. Для каждого элемента списка освобождение памяти

test_memory_fragmentation(int initial_fill, int remove_count, int additional_fill) - тест фрагментации памяти:

1. Инициализация очереди и заполнение initial_fill элементами

2. Вывод состояния очереди после заполнения

3. Удаление remove_count элементов с сохранением данных

4. Вывод состояния очереди после удаления и списка освобожденных областей

5. Добавление обратно сохраненных элементов

6. Вывод состояния очереди после повторного добавления и списка освобожденных областей

7. Анализ повторного использования освобожденной памяти

read_int(int *num) - чтение целого числа:

1. Чтение строки заданной длины

2. Проверка, что строка не пустая

3. Проверка, что строка содержит только цифры и знак

4. Преобразование строки в число с помощью sscanf

5. Возврат кода успеха или ошибки

read_double(double *num) - чтение вещественного числа:

1. Чтение строки заданной длины

2. Проверка, что строка не пустая

3. Проверка, что строка содержит только цифры, знак и одну точку

4. Преобразование строки в число с помощью sscanf

5. Возврат кода успеха или ошибки

menu_print(void) - вывод меню программы:

1. Вывод заголовка программы

2. Вывод пунктов меню с номерами и описаниями

3. Вывод приглашения для ввода

print_queue(queue_t *queue) - вывод очереди (общая функция):

1. Определение типа очереди (ARRAY_QUEUE или LIST_QUEUE)

2. Вызов соответствующей функции вывода (print_arr_queue или print_list_queue)

queue_print_free_area(queue_t *queue) - вывод свободных областей памяти:

1. Проверка, что очередь реализована списком

2. Вызов функции print_free_area для очереди-списка

print_measurements(void) - вывод замеров производительности:

1. Определение тестовых размеров очереди (1, 10, 50, 100, 335, 500)

2. Вывод заголовка таблицы

3. Для каждого размера:

- Создание тестовой последовательности заявок
- Замер времени операций для массива и списка
- Расчет использования памяти
- Вывод строки таблицы

4. Форматированный вывод таблицы

simulation_queue(queue_t *queue1, queue_t *queue2, double t1_start, double t1_end, double t2_start, double t2_end, double probability) - обслуживания системы обслуживания:

1. Инициализация обслуживающих аппаратов и переменных статистики

2. Заполнение первой очереди 100 начальными заявками

3. Цикл до обработки 1000 заявок во втором аппарате:

- Обработка заявок в ОА1 и ОА2 при свободных аппаратах и непустых очередях

- Определение ближайшего события завершения обработки
- Обновление времени простоя ОА2
- Обновление текущего времени системы
- Обработка завершенной заявки:

Для ОА1: с вероятностью probability возврат в первую очередь, иначе во вторую

Для ОА2: добавление в первую очередь и увеличение счетчика обработанных

- Каждые 100 заявок вывод промежуточной статистики

4. Вывод итоговой статистики (время, простоя, количество срабатываний, средние длины)

queue_init(queue_t *queue, queue_type_t queue_type, int max_size) - инициализация очереди (общая функция):

1. Установка типа очереди
2. Вызов соответствующей функции инициализации (arr_queue_init или list_queue_init)

is_queue_empty(queue_t *queue) - проверка очереди на пустоту (общая функция):

1. Определение типа очереди
2. Вызов соответствующей функции проверки пустоты

is_queue_full(queue_t *queue) - проверка очереди на переполнение (общая функция):

1. Определение типа очереди
2. Вызов соответствующей функции проверки переполнения

enqueue(queue_t *queue, request_t elem) - добавление элемента в очередь (общая функция):

1. Проверка, что очередь не переполнена

2. Определение типа очереди

3. Вызов соответствующей функции добавления

4. Возврат кода успеха или ошибки

dequeue(queue_t *queue, request_t *removed_elem) - удаление элемента из очереди (общая функция):

1. Проверка, что очередь не пуста

2. Определение типа очереди

3. Вызов соответствующей функции удаления

4. Возврат кода успеха или ошибки

queue_length(queue_t *queue) - получение длины очереди:

1. Определение типа очереди

2. Возврат поля size соответствующей структуры очереди

queue_free(queue_t *queue) - освобождение памяти очереди:

1. Проверка, что очередь реализована списком

2. Вызов функции free_list_queue для освобождения памяти

Тесты

Позитивные тесты

1) **Изменение параметров на массиве, общее время работы получается результатом обработки 1000 заявок аппарата ОА2**

P=0.1 возврата в ОА1

Ожидаемое время 4.5*1000=4500

Итоги симуляции:

Общее время работы: 4452.39

Теоретический рассчёт: 4500.00

Процент погрешности: 1.06%

Время простоя ОА2: 5.04

Количество срабатываний ОА1: 1227

Средняя длина первой очереди 14.40

Средняя длина второй очереди: 84.90

2) **Изменение параметров на списке, общее время работы получается результатом обработки 1000 заявок аппарата ОА2**

P=0.1 возврата в ОА1

Ожидаемое время 4.5*1000=4500

Итоги симуляции:

Общее время работы: 4431.16

Теоретический рассчёт: 4500.00

Процент погрешности: 1.53%

Время простоя ОА2: 0.82

Количество срабатываний ОА1: 1220

Средняя длина первой очереди 13.00

Средняя длина второй очереди: 86.40

3) Простой аппарата ОА2 на массиве, так как ОА1 медленный

ОА1: 0-10 (ср. 5.0)

ОА2: 1-3 (ср. 2.0)

P=0.7

N1=3333.33

Ожидаемое время max(5×3333.33, 2×1000)=max(16666.67,2000)=16667

Простой ОА2: 14667

Итоги симуляции:

Общее время работы: 16909.24

Теоретический рассчёт: 16940.00

Процент погрешности: 0.18%

Время простоя ОА2: 14913.50

Количество срабатываний ОА1: 3388

Средняя длина первой очереди 98.80

Средняя длина второй очереди: 0.20

4) Простой аппарата ОА2 на списке, так как ОА1 медленный

ОА1: 0-10 (ср. 5.0)

ОА2: 1-3 (ср. 2.0)

P=0.7

N1=3333.33

Ожидаемое время max(5×3333.33, 2×1000)=max(16666.67,2000)=16667

Простой ОА2: 14667

Итоги симуляции:

Общее время работы: 16507.05

Теоретический рассчёт: 16570.00

Процент погрешности: 0.38%

Время простоя ОА2: 14548.33

Количество срабатываний ОА1: 3314

Средняя длина первой очереди 98.90

Средняя длина второй очереди: 0.10

5) **Моделирование очереди на массиве, параметры по умолчанию**

среднее время ОА1=3 среднее время ОА2 =4.5

Ожидаемое время max(3*3.3*1000, 4.5*1000)=10000

Итоги симуляции:

Общее время работы: 10217.18

Теоретический рассчёт: 10167.00

Процент погрешности: 0.49%

Время простоя ОА2: 5779.77

Количество срабатываний ОА1: 3389

Средняя длина первой очереди 97.90

Средняя длина второй очереди: 1.10

6) **Моделирование очереди на списке, параметры по умолчанию**

Вводимые данные — параметры по умолчанию

среднее время ОА1=3 среднее время ОА2 =4.5

Ожидаемое время max(3*3.3*1000, 4.5*1000)=10000

Итоги симуляции:

Общее время работы: 10300.17

Теоретический рассчёт: 10383.00

Процент погрешности: 0.80%

Время простоя ОА2: 5834.91

Количество срабатываний ОА1: 3461

Средняя длина первой очереди 98.40

Средняя длина второй очереди: 0.60

7) Фрагментация

Освобождается элемент. Адрес: 0x602caa5c5990, ID: 87

Освобождается элемент. Адрес: 0x602caa5ccb30, ID: 83

Добавлен элемент. Адрес: 0x602caa5ccb30, ID: 83

Добавлен элемент. Адрес: 0x602caa5d2af0, ID: 87

Присутствует

8) почти нет простоя у аппарата ОА2, так как ОА1 быстрый

ОА1: 0-4 (ср. 2.0)

ОА2: 1-6 (ср. 3.5)

p=0.3

avg_t1=2, avg_t2=3.5

p_to_Q2=0.7

N1=1000/0.7≈1428.57

Ожидаемое время max(2×1428.57, 3.5×1000)=max(2857.14,3500)=3500

Простой ОА2: 0

Итоги симуляции:

Общее время работы: 3526.39

Теоретический рассчёт: 3500.00

Процент погрешности: 0.75%

Время простоя ОА2: 2.93

Количество срабатываний OA1: 1591

Средняя длина первой очереди 13.10

Средняя длина второй очереди: 86.40

9) Сравнение эффективности — таблица с результатами измерений(представлена в оценке эффективности)

Негативные тесты

Описание	Ожидаемый результат
Ввод несуществующего пункта меню:100	Сообщение "Выберите 1 из пунктов меню"
Ввод буквенных символов в пункты меню: a	Сообщение "Ошибка: Пожалуйста, введите число."
Ввод отрицательного времени обработки: -100	Сообщение "Ошибка ввода времени. Попробуйте еще раз."
Ввод некорректного диапазона времени: 10 1	Сообщение "Ошибка ввода времени. Попробуйте еще раз."
Ввод вероятности вне диапазона [0,1]: 11	Сообщение "Ошибка ввода вероятности. Попробуйте еще раз."

Оценка эффективности (10000 итераций замера)

Добавление элемента

Размер	Массив, мс	Список, мс	Эффективность, %
1	0.0636	0.0903	29,6
10	0.1628	0.5426	70
50	0.9065	4.5778	80,2
100	0.8766	5.6547	84,5
335	2.9721	11.3672	73,9
500	4.3961	16.4903	73,3

Удаление элемента

Размер	Массив, мс	Список, мс	Эффективность,%
1	0.0479	0.1020	53
10	0.1655	0.6817	75,7
50	0.4968	1.0505	52,7
100	0.8490	2.0451	58,5
335	2.7852	6.0238	53,8
500	4.1400	8.9041	53,5

Память

Размер	Массив, байт	Список, байт	Эффективность,%
1	8000	24	99,7
10	8000	240	97
50	8000	1200	85
100	8000	2400	70
335	8000	8040	-0,5
500	8000	12000	-50

Вывод

В ходе выполнения лабораторной работы было проведено сравнение эффективности реализации стека на основе массива и односвязного списка. Анализ выполнялся для различных размеров стека: 10, 100 и 1000 элементов.

Результаты исследования демонстрируют значительное преимущество массива по скорости выполнения операций. Для стека размером 1000 элементов массив оказался быстрее на 97.35%, при размере 100 элементов - на 92.15%, а для 10 элементов - на 85.67%. Наиболее заметная разница наблюдается в операции проверки на палиндром, где массив превосходит список на 91.5-97.5% в зависимости от размера.

Что касается использования памяти, ситуация зависит от степени заполнения стека. При полном заполнении (1000 элементов) массив экономичнее на 93.7%, поскольку список требует дополнительной памяти для хранения указателей. Однако при малом заполнении (10 элементов) список становится экономичнее на 84.1%, так как массив всегда занимает фиксированный объем памяти независимо от фактического количества элементов.

Таким образом, реализация стека массивом предпочтительна в случаях, когда известен максимальный размер стека и требуется высокая производительность операций. Реализация списком целесообразна при работе с стеками переменного размера или когда важна экономия памяти при неполном заполнении. Выбор конкретной реализации должен основываться на требованиях конкретной задачи к производительности и использованию памяти.

Контрольные вопросы

1. Что такое FIFO и LIFO?

FIFO (First In – First Out) – принцип «первым пришёл – первым вышел», используемый в очередях. Элементы добавляются в «хвост» и извлекаются из «головы».

LIFO (Last In – First Out) – принцип «последним пришёл – первым вышел», используемый в стеках. Элементы добавляются и извлекаются с одного конца..\

2. Каким образом, и какой объем памяти выделяется под хранение очереди при различной ее реализации?

Реализация в виде массива: выделяется статическая непрерывная область памяти размером m * L байт, где m – максимальное количество элементов, L – размер одного элемента. Память фиксирована и выделяется заранее.

Реализация в виде линейного списка: память выделяется динамически под каждый новый элемент. Объём ограничен только доступной оперативной памятью. Каждый узел содержит информационное поле и указатель на следующий элемент.

3. Каким образом освобождается память при удалении элемента из очереди при ее различной реализации?

При хранении очереди связанным списком, верхний элемент удаляется путём освобождения памяти для него и смещения указателя, указывающего на начало стека. При удалении из очереди, реализованной массивом, смещается лишь указатель на голову очереди, а сами данные остаются в памяти, но считаются «стёртыми».

4. Что происходит с элементами очереди при ее просмотре?

При просмотре очереди, реализованной на массиве: осуществляется последовательное обращение к элементам массива по индексам, начиная с индекса головы и завершая индексом хвоста. Во время этого процесса данные только читаются, а сами элементы остаются в массиве без изменений — никакого удаления или перемещения не происходит.

При просмотре очереди, реализованной в виде списка: выполняется последовательный обход узлов списка, начиная с головного элемента и следуя по цепочке указателей next до достижения хвостового элемента. Каждый узел доступен только для чтения, его связи и данные не изменяются, элементы не удаляются из списка.

5. От чего зависит эффективность физической реализации очереди?

Эффективность реализации очереди массивом или списком зависит от ресурсов памяти и начального размера очереди. Если нам важна занимаемая очередью память и скорость обработки, то эффективнее использовать очередь на массиве, а если мы не знаем сколько элементов может храниться в очереди, то лучше использовать очередь на списке.

6. Каковы достоинства и недостатки различных реализаций очереди в зависимости от выполняемых над ней операций?

Массив (кольцевая очередь):

- Достоинства: быстрота операций, отсутствие фрагментации, кэш-эффективность.
- Недостатки: фиксированный размер, возможна избыточная память, возможное переполнение(необходимо отслеживать).

Односвязный список:

- Достоинства: динамическое изменение размера, эффективное использование памяти при неполном заполнении.
- Недостатки: медленнее операций, возможна фрагментация, дополнительные затраты на хранение указателей.

7. Что такое фрагментация памяти, и в какой части ОП она возникает?

Фрагментация памяти – явление, при котором в памяти образуются небольшие разрозненные свободные участки, которые невозможно использовать для размещения данных, даже если суммарный объём свободной памяти достаточен. Возникает в *куче (heap)* при динамическом выделении и освобождении памяти разного размера.

8. **Для чего нужен алгоритм «близнецов»?**

Алгоритм «близнецов» используется для борьбы с фрагментацией памяти. Память делится на блоки размером степени двойки. При запросе выделяется блок минимально достаточного размера. При освобождении блоки объединяются с соседними свободными блоками того же размера («близнецами»), что уменьшает фрагментацию

9. Какие дисциплины выделения памяти вы знаете?

- *First-Fit* – выделение первого подходящего по размеру блока.
- *Best-Fit* – выделение наиболее подходящего (близкого к запрошенному размеру) блока.
- *Worst-Fit* – выделение наибольшего свободного блока.
- *Buddy system* (система «близнецов») – выделение блоков размером степени двойки.

10. На что необходимо обратить внимание при тестировании программы?

Нужно обратить внимание на корректное выделение и освобождение памяти, а также отслеживать переполнение или пустоту очереди.

11. Каким образом физически выделяется и освобождается память при динамических запросах?

Когда программа запрашивает память, операционная система ищет подходящий свободный блок и регистрирует его в таблице занятых областей памяти. При освобождении памяти, операционная система удаляет информацию о этом блоке из этой таблицы